



Bases de Datos

Clase 9

Transacciones y Locks

Hasta ahora

Hasta este punto, el curso se ha centrado exclusivamente en la **base de datos**:



Diseño



Estructura lógica



Lenguaje de
consulta SQL

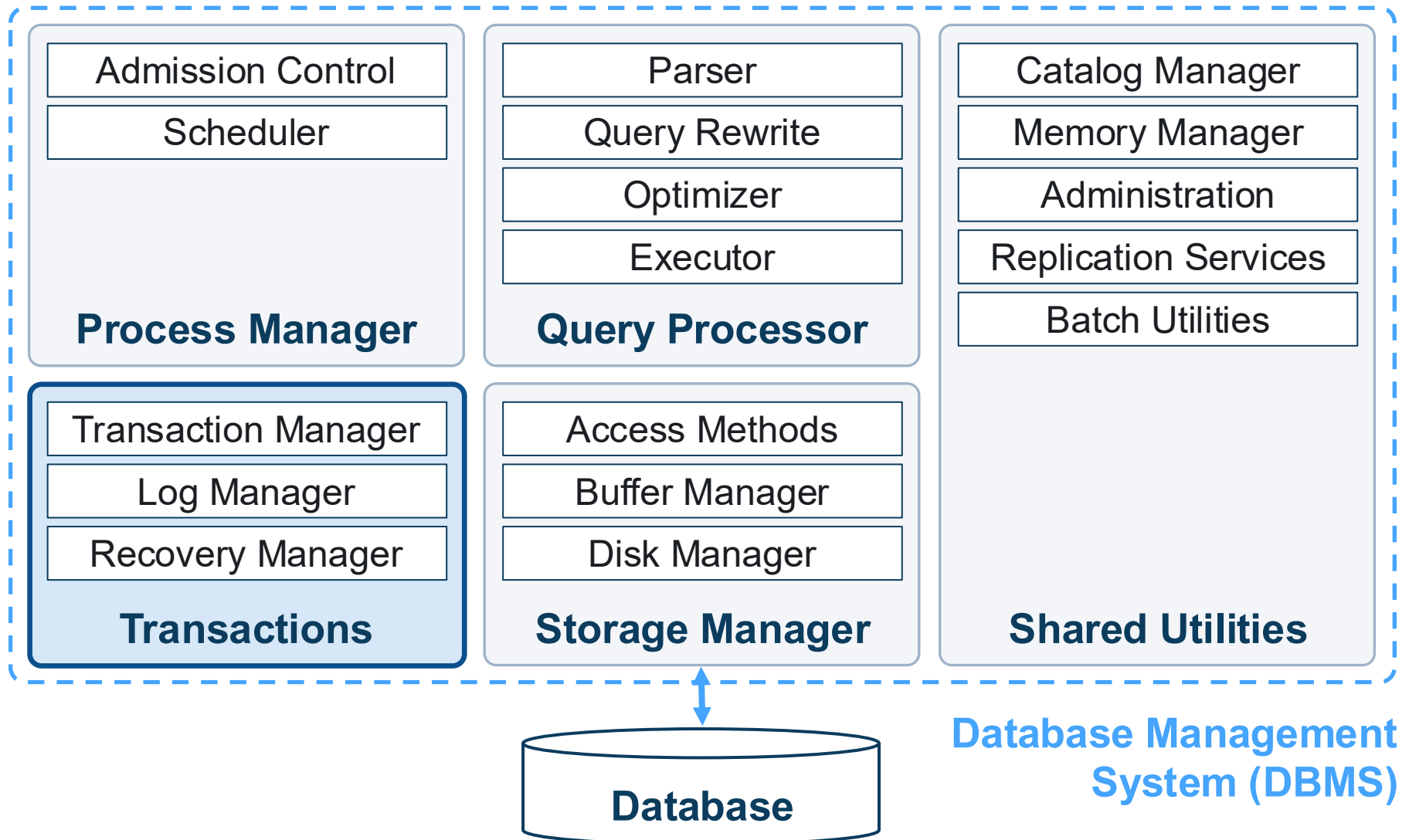


¿Qué ocurre con el DBMS?

A partir de esta clase, el estudio se enfoca en el sistema de gestión de bases de datos.

DBMS

Arquitectura general





Transacciones

¿Qué es una transacción?

Una **transacción** es una secuencia de una o más **operaciones** que modifican o consultan la base de datos. Algunos ejemplos:



Transferencias
de dinero entre
cuentas



Compras por
internet



Inscripción de un
curso

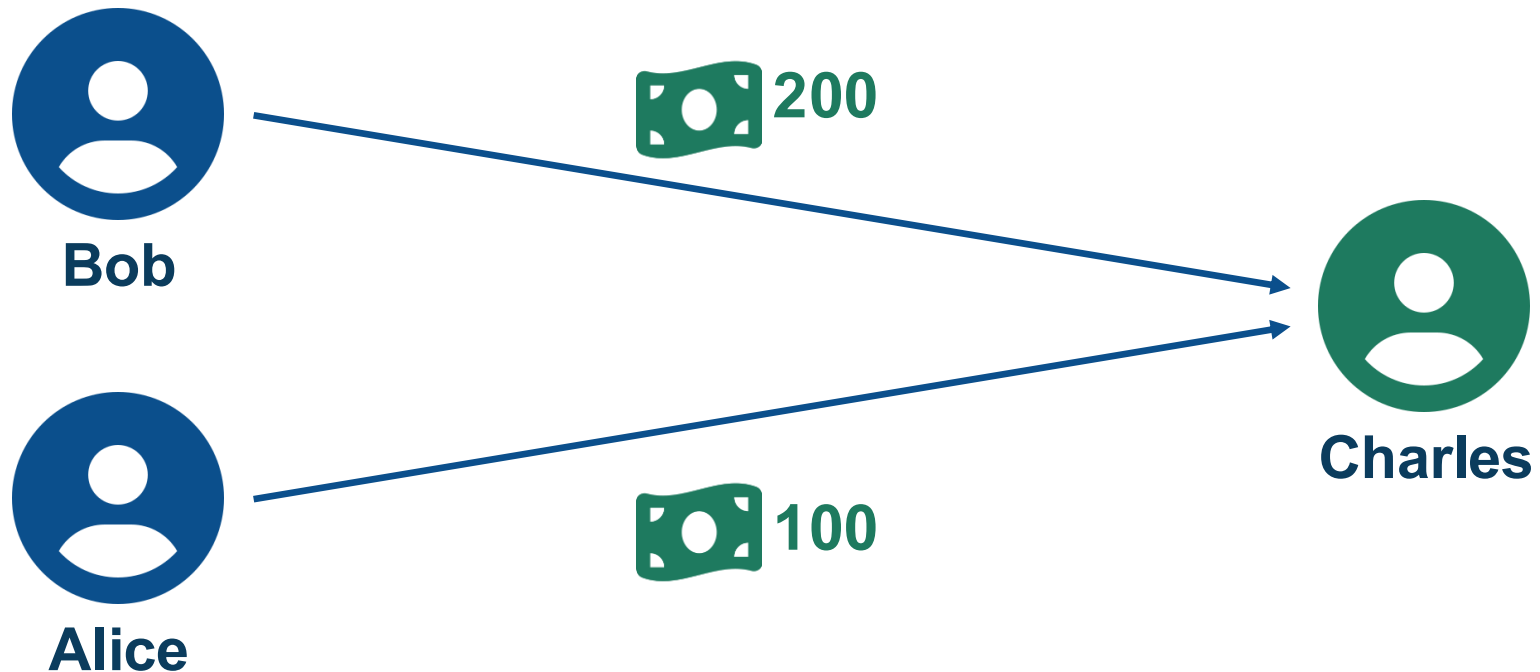


El propósito principal de las transacciones es **garantizar la integridad de los datos** y la **consistencia del estado** de la base de datos, incluso en presencia de fallos y errores.

Motivación

Alice y Bob comparten una cuenta bancaria y cada uno realiza una transferencia a Charles:

- Alice transfiere **100** a su amigo Charles.
- Bob transfiere **200** a su amigo Charles.



Motivación

Ejecución secuencial



Cuenta A & B (Alice y Bob)

1000 → 900 → 700



Cuenta C (Charles)

1000 → 1100 → 1300

Proceso Alice	Proceso Bob	Saldo A & B	Saldo C
READ(saldoAB, x)		1000	
WRITE(saldoAB, x - 100)		900	
READ(saldoC, x)			1000
WRITE(saldoC, x + 100)			1100
	READ(saldoAB, y)	900	
	WRITE(saldoAB, y - 200)	700	
	READ(saldoC, y)		1100
	WRITE(saldoC, y + 200)		1300

Acceso concurrente

Opción 1

Sin embargo, el acceso a la base de datos es **concurrente**: las operaciones de ambos procesos pueden intercalarse.

Proceso Alice	Proceso Bob	Saldo A & B	Saldo C
READ(saldoAB, x)		1000	
WRITE(saldoAB, x - 100)		900	
	READ(saldoAB, y)	900	
	WRITE(saldoAB, y - 200)	700	
	READ(saldoC, y)		1000
	WRITE(saldoC, y + 200)		1200
READ(saldoC, x)			1200
WRITE(saldoC, x + 100)			1300

Acceso concurrente

Opción 2

Proceso Alice	Proceso Bob	Saldo A & B	Saldo C
READ(saldoAB, x)		1000	
WRITE(saldoAB, x - 100)		900	
READ(saldoC, x)			x = 1000
	READ(saldoAB, y)	900	
	WRITE(saldoAB, y - 200)	700	
	READ(saldoC, y)		1000
	WRITE(saldoC, y + 200)		1200
WRITE(saldoC, x + 100)			1100



¡Se perdieron 200!

¿Qué está pasando?

Las operaciones de cada depósito se **intercalan**.

El ideal

Cada depósito se ejecuta en el orden en que fue solicitado.

Lo real

Para optimizar los accesos a disco, conviene intercalar las operaciones.

El DBMS intercala las operaciones para optimizar la ejecución y, al mismo tiempo, **preserva las transacciones.**

¿Solución?

Agrupar las operaciones de modo que se ejecuten como una sola unidad: **se ejecuta entera o no se ejecuta.**

Proceso Alice	Proceso Bob	Saldo A & B	Saldo C
Comienzo transacción T1		1000	1000
READ(saldoAB, x)			
WRITE(saldoAB, x - 100)			
READ(saldoC, x)			
WRITE(saldoC, x + 100)			
Fin T1		900	1100
	Comienzo transacción T2	900	1100
	READ(saldoAB, y)		
	WRITE(saldoAB, y - 200)		
	READ(saldoC, y)		
	WRITE(saldoC, y + 200)		
	Fin T2	700	1300

Transacciones concurrentes

¿Y si todas las transacciones se ejecutan a la vez?

Proceso Alice	Proceso Bob	Saldo A & B	Saldo C
Comienzo transacción T1	Comienzo transacción T2	1000	1000
READ(saldoAB, x)	READ(saldoAB, y)		
WRITE(saldoAB, x - 100)	WRITE(saldoAB, y - 200)		
READ(saldoC, x)	READ(saldoC, y)		
WRITE(saldoC, x + 100)	WRITE(saldoC, y + 200)		
Fin T1	Fin T2	700	1300

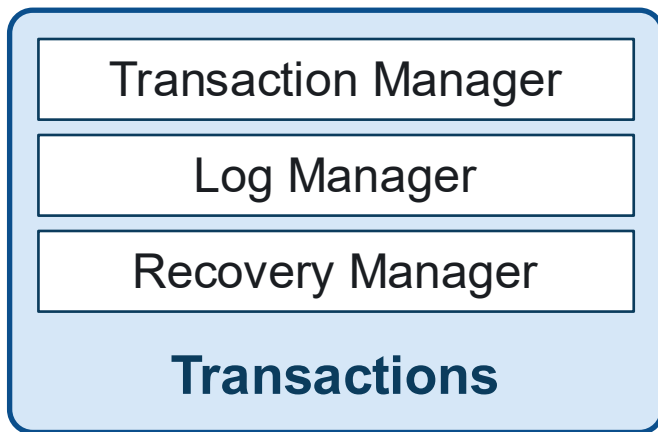
¿Entonces?



ACID

Transactions

Componentes del DBMS



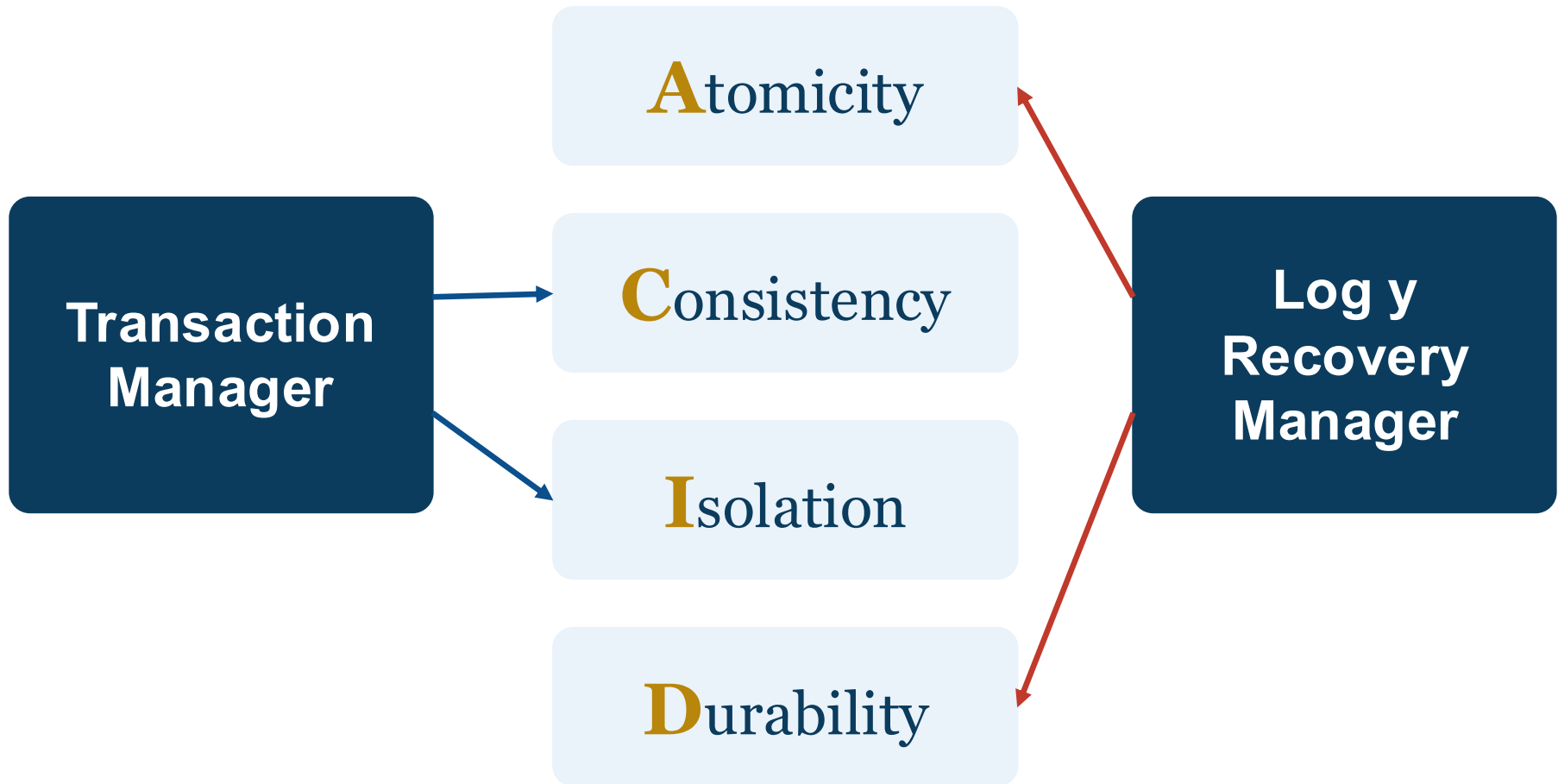
Transaction Manager

Se encarga de asegurar
Isolation y Consistency.

Log y Recovery Manager

Se encargan de asegurar
Atomicity y Durability.

Asegurar las propiedades ACID



Sobre Transactions



Uno de los componentes fundamentales de un DBMS.



Fundamental para aplicaciones que requieren seguridad (CIA).



Objeto de uno de los **Premios Turing** en Bases de Datos.

Transaction Manager

Log Manager

Recovery Manager

Transactions

Propiedades ACID



Atomicity

O se ejecutan todas las operaciones de la transacción, o no se ejecuta ninguna.

Consistency

Cada transacción preserva la consistencia de la BD (restricciones de integridad, etc.).

Isolation

Cada transacción debe ejecutarse como si se ejecutara sola, de forma aislada.

Durability

Los cambios realizados por cada transacción son permanentes, independientemente de cualquier tipo de falla.

¿Qué ocurre sin ACID?

Caso: corte de luz a mitad de una transacción

Sin Atomicity y Durability

- La BD vuelve a su estado anterior, pero la transacción se pierde.
- Un cambio de la transacción no queda reflejado en la BD.

Sin Consistency

- La BD viola momentáneamente las restricciones.
- Tras ejecutar una transacción, la BD no cumple las restricciones.

Sin Isolation

- El DBMS planifica el orden de las operaciones.
- El resultado difiere del de ejecutar las transacciones en serie.

Conflictos entre transacciones

Lectura sucia

Write - Read

Una transacción accede a datos modificados por otra transacción que aún no ha terminado.

Lectura no repetible

Read - Write

Una fila se lee dos veces y su valor cambia entre ambas lecturas, porque otra transacción la modifica en ese intervalo.

Actualización perdida

Write - Write

Dos transacciones actualizan la misma fila en un intervalo tal que una actualización sobrescribe a la otra.

Definición de operaciones

Read(X) o R(X)

Lectura del elemento X

Write(X) o W(X)

Escritura del elemento X

Abort o A

Abortar la transacción

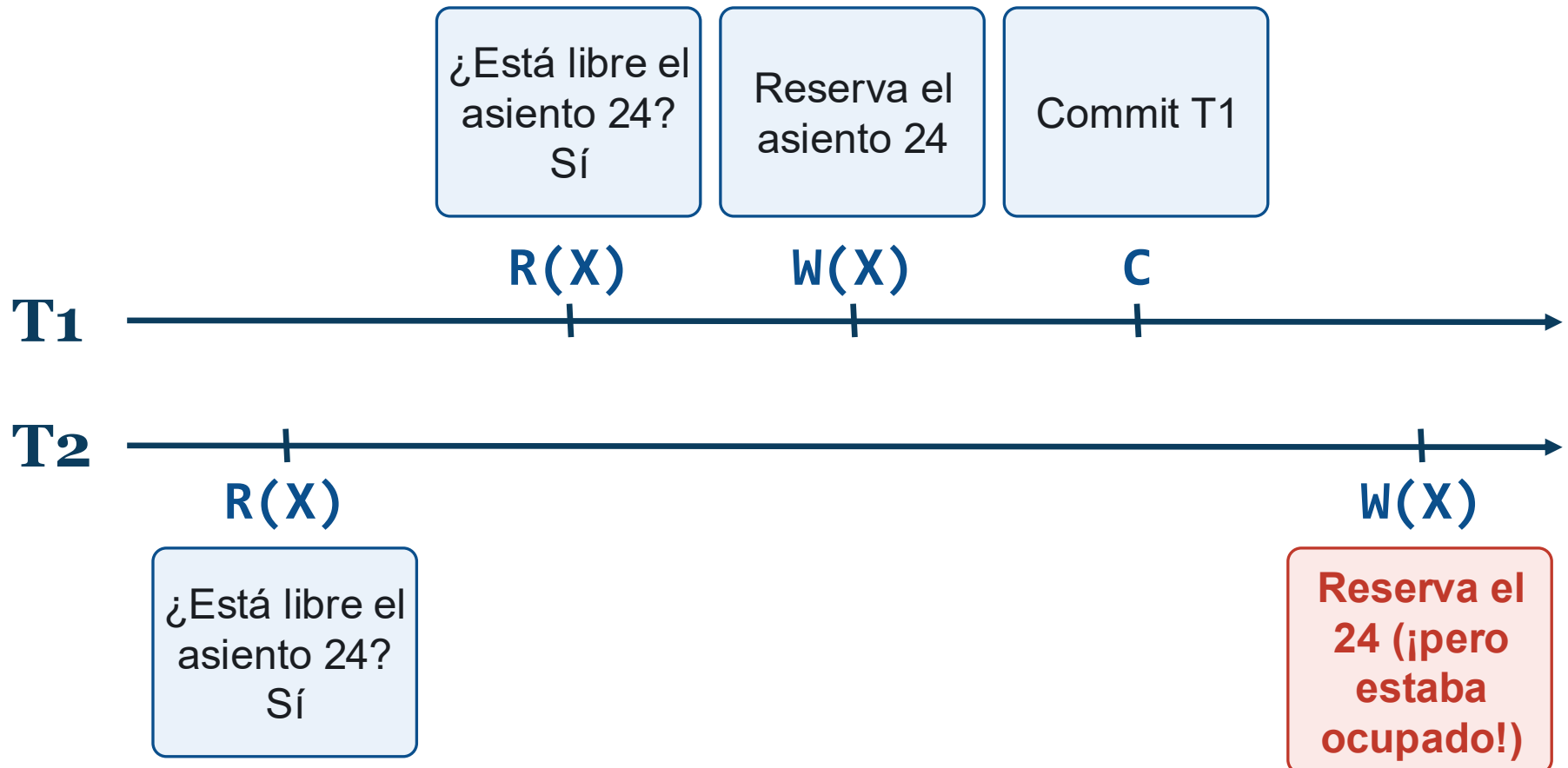
Commit o C

Finalizar la transacción

Actualización perdida (WW)

Ejemplo

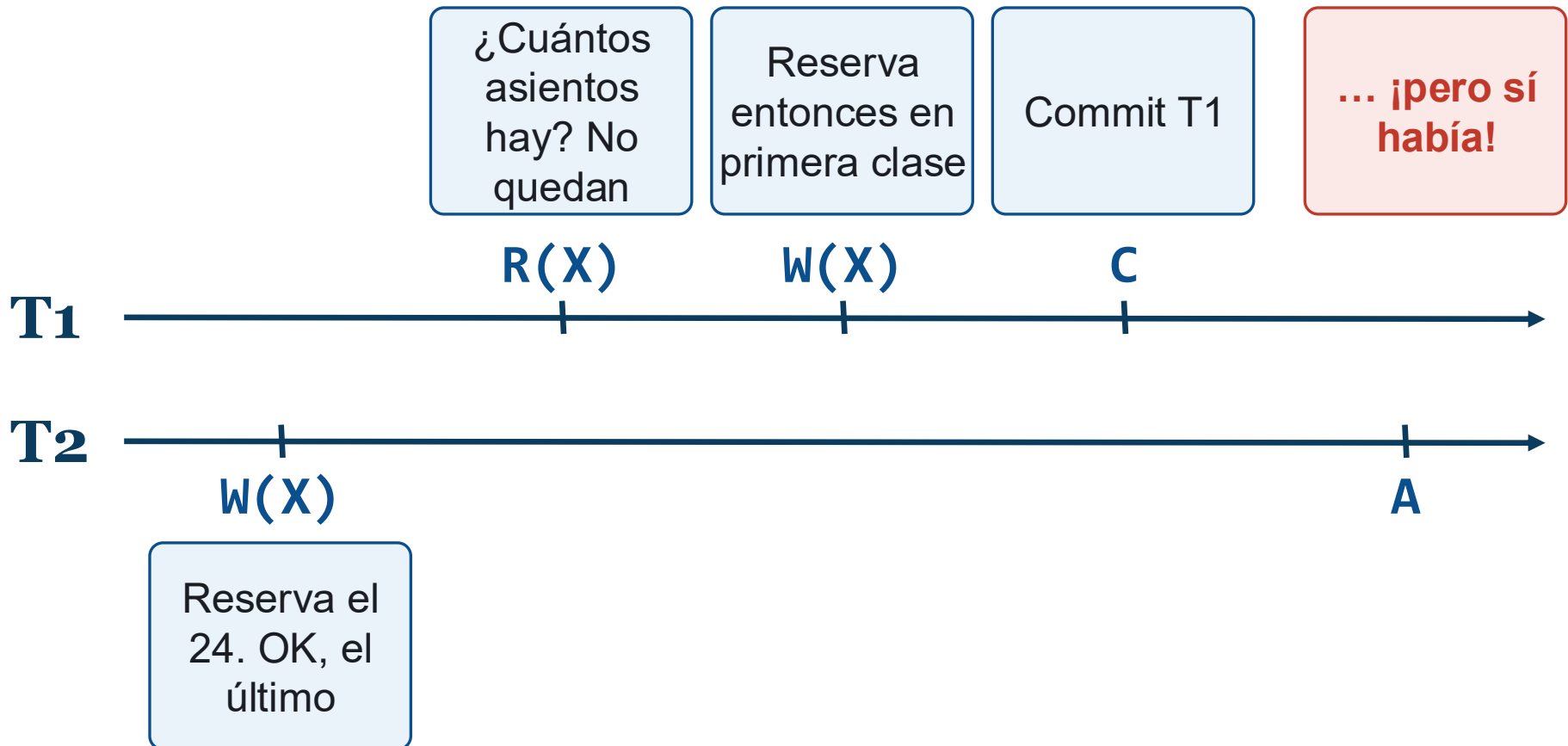
“Una transacción sobrescribe los datos que otra ya había escrito.”



Lectura sucia (WR)

Ejemplo

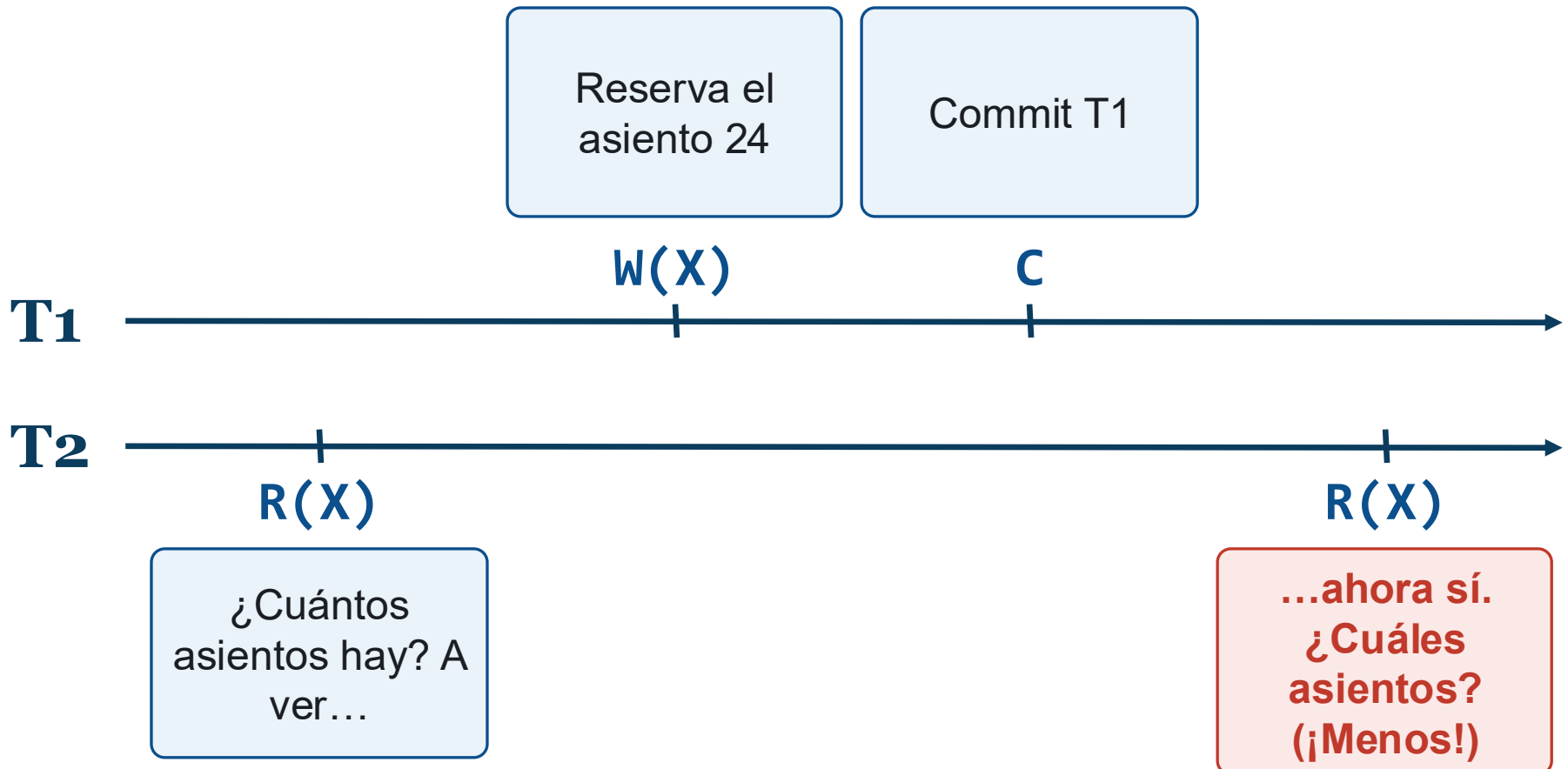
“Una transacción lee lo que otra escribió, pero que aún no se había confirmado.”



Lectura no repetible (RW)

Ejemplo

“Una transacción sobrescribe un dato que otra ya había leído, pero aún no había confirmado.”





¿Qué reglas sigue entonces el DBMS?

Schedules

¿Qué es un schedule?

1 =
2 =

Un **schedule S** es una **secuencia de operaciones primitivas** de una o más transacciones, tal que, para toda transacción, sus acciones aparecen en el mismo orden que en su definición.

Schedule

Transacciones de un schedule

T1	T2
READ(A,x)	READ(A,y)
$x := x + 100$	$y := y * 2$
WRITE(A,x)	WRITE(A,y)
READ(B,x)	READ(B,y)
$x := x + 200$	$y := y * 3$
WRITE(B,x)	WRITE(B,y)

¿Son equivalentes?

Schedule 1

T1	T2
READ(A,x)	
x := x + 100	
WRITE(A,x)	
READ(B,x)	
x := x + 200	
WRITE(B,x)	
	READ(A,y)
	y := y * 2
	WRITE(A,y)
	READ(B,y)
	y := y * 3
	WRITE(B,y)

Schedule 2

T1	T2
READ(A,x)	
x := x + 100	
WRITE(A,x)	
	READ(A,y)
	y := y * 2
	WRITE(A,y)
READ(B,x)	
x := x + 200	
WRITE(B,x)	
	READ(B,y)
	y := y * 3
	WRITE(B,y)

Schedule 2

Otro schedule

T1	T2
READ(A,x)	
x := x + 100	
WRITE(A,x)	
	READ(A,y)
	y := y * 2
	WRITE(A,y)
READ(B,x)	
x := x + 200	
WRITE(B,x)	
	READ(B,y)
	y := y * 3
	WRITE(B,y)

Schedule serial

Un schedule serial

Un **schedule S** es **serial** si no hay intercalación entre las acciones de sus transacciones.

T1	T2
READ(A,x)	
x := x + 100	
WRITE(A,x)	
READ(B,x)	
x := x + 200	
WRITE(B,x)	
	READ(A,y)
	y := y * 2
	WRITE(A,y)
	READ(B,y)
	y := y * 3
	WRITE(B,y)

Schedule serializable

Un **schedule S** es **serializable** si existe algún schedule **S'** serial con las mismas transacciones, tal que el resultado de **S** y **S'** es el mismo para todo estado inicial de la BD.

Las operaciones sobre A (en x e y) no alteran el orden de x e y sobre B; por lo tanto, **S es serializable**.

T1	T2
READ(A,x)	
$x := x + 100$	
WRITE(A,x)	
	READ(A,y)
	$y := y * 2$
	WRITE(A,y)
READ(B,x)	
$x := x + 200$	
WRITE(B,x)	
	READ(B,y)
	$y := y * 3$
	WRITE(B,y)

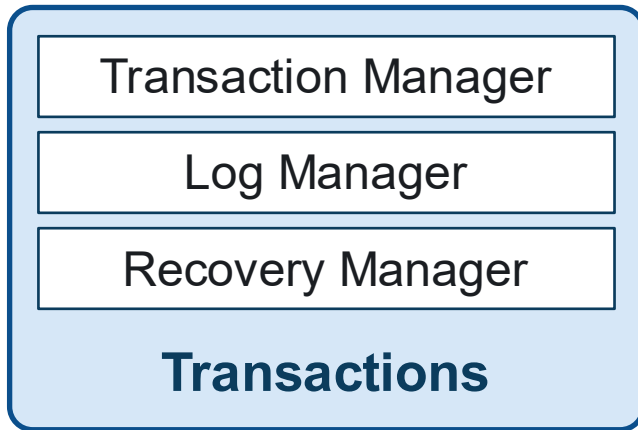
Schedule no serializable

T1	T2
READ(A,x)	
$x := x + 100$	
WRITE(A,x)	
	READ(A,y)
	$y := y * 2$
	WRITE(A,y)
	READ(B,y)
	$y := y * 3$
	WRITE(B,y)
READ(B,x)	
$x := x + 200$	
WRITE(B,x)	

En T2, `READ(B,y)` y las operaciones siguientes modifican B antes de `READ(B,x)` de T1.

Si el orden cambia, cambian los resultados; por lo mismo, **el schedule no es serializable.**

Transacciones



La tarea del **Transaction Manager** es permitir solo schedules que sean **serializables**.



¿Qué problemas pueden surgir si un schedule no es serializable?

Posibles problemas

Lo que se espera

T1	T2
READ(A,x)	
x := x + 100	
WRITE(A,x)	
	READ(A,y)
	y := y + 100
	WRITE(A,y)

Lo que ejecuta el sistema

T1	T2
READ(A,x)	
	READ(A,y)
x := x + 100	
	y := y + 100
WRITE(A,x)	
	WRITE(A,y)

Notación

Si la transacción i ejecuta $READ(X, t)$, se escribe:

$$READ(X, t) \rightarrow R_i(X)$$

Si la transacción i ejecuta $WRITE(X, t)$, se escribe:

$$WRITE(X, t) \rightarrow W_i(X)$$

Acciones no conflictivas

Las siguientes acciones son **no conflictivas** para dos transacciones distintas i, j :

$$R_i(X), R_j(Y)$$

$$R_i(X), W_j(Y) \quad \text{con } X \neq Y$$

$$W_i(X), R_j(Y) \quad \text{con } X \neq Y$$

$$W_i(X), W_j(Y) \quad \text{con } X \neq Y$$



Pueden intercambiarse de orden en un schedule.

Permutaciones permitidas

T1	T2
READ(A)	
	READ(A)
	READ(C)
READ(B)	
WRITE(A)	
	WRITE(C)
READ(D)	



Permutaciones permitidas

T1	T2
	READ(A)
READ(A)	
	READ(C)
READ(B)	
WRITE(A)	
	WRITE(C)
READ(D)	



Permutaciones permitidas

T1	T2
	READ(A)
READ(A)	
READ(B)	
	READ(C)
WRITE(A)	
	WRITE(C)
READ(D)	

Permutaciones permitidas

T1	T2
	READ(A)
READ(A)	
READ(B)	
WRITE(A)	
	READ(C)
	WRITE(C)
READ(D)	



Permutaciones permitidas

T1	T2
	READ(A)
READ(A)	
READ(B)	
WRITE(A)	
	READ(C)
READ(D)	
	WRITE(C)

Acciones conflictivas

Las siguientes acciones son **conflictivas** para dos transacciones distintas i, j :

Lectura – Escritura
 $R_i(X), W_j(X)$

La transacción i lee X y la transacción j escribe X .

Escritura – Lectura
 $W_i(X), R_j(X)$

La transacción i escribe X y la transacción j lee X .

Escritura – Escritura
 $W_i(X), W_j(X)$

Ambas transacciones, i y j , escriben en X .



¡No es posible cambiar su orden en un schedule a la ligera!

Permutaciones no permitidas

T1	T2
READ(A)	
	WRITE(A)
	WRITE(C)
WRITE(C)	
WRITE(A)	
	READ(A)
READ(D)	



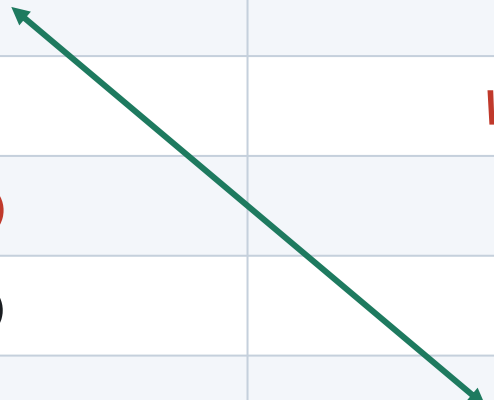
Permutaciones no permitidas

T1	T2
	WRITE(A)
READ(A)	
	WRITE(C)
WRITE(C)	
WRITE(A)	
	READ(A)
READ(D)	

Permutaciones no permitidas

¡Cuidado!

T1	T2
	WRITE(A)
READ(A)	
	WRITE(C)
WRITE(C)	
WRITE(A)	
	READ(A)
READ(D)	



Permutaciones no permitidas

¡Cuidado!

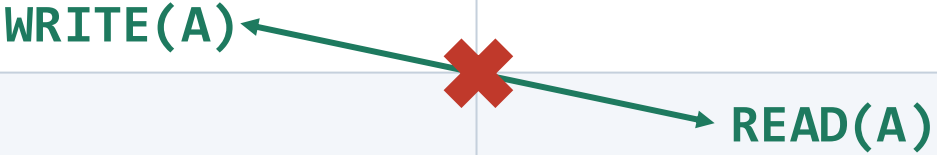
T1	T2
	WRITE(A)
READ(A)	
	WRITE(C)
WRITE(C)	
WRITE(A)	
	READ(A)
READ(D)	



Permutaciones no permitidas

¡Cuidado!

T1	T2
	WRITE(A)
READ(A)	
	WRITE(C)
WRITE(C)	
WRITE(A)	
	READ(A)
READ(D)	



Acciones no conflictivas

Un par de operaciones **consecutivas** puede permutarse si:



No usan el mismo recurso.



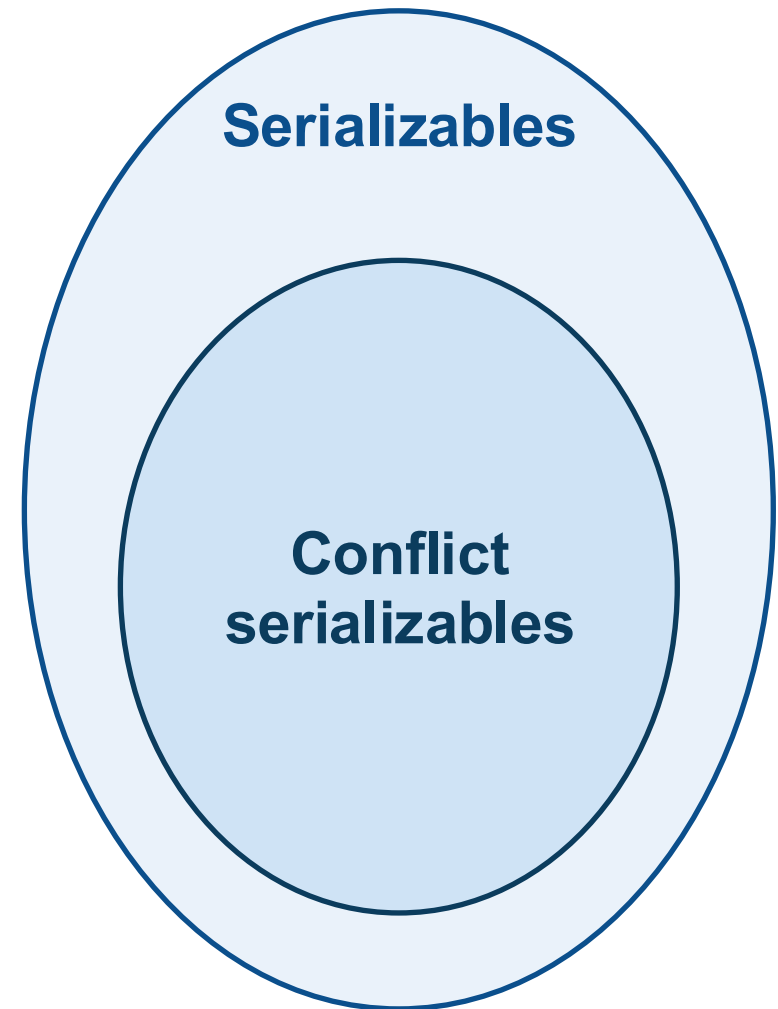
Usan el mismo recurso,
pero **ambas son de lectura.**

$R_i(X), R_j(X)$ ✓

Conflict serializable

Un schedule es **conflict serializable** si es posible transformarlo en uno serial mediante permutaciones.

Si un schedule es *conflict serializable*, entonces también es *serializable*; pero existen schedules serializables que no son *conflict serializable*.



Conflict serializable

Mediante este proceso de permutaciones:



Se lleva el schedule a uno **serial**.



Se preserva el orden de **todos** los conflictos.

Permutando a serial

Ejemplo

¿Es serializable? Se procede a permutar:

T1	T2	T3
	$R_2(A)$	
$R_1(B)$		
	$W_2(A)$	
		$R_3(A)$
$W_1(B)$		
		$W_3(A)$
	$R_2(B)$	
	$W_2(B)$	



Permutando a serial

Ejemplo

Se procede a permutar:

T1	T2	T3
$R_1(B)$		
	$R_2(A)$	
	$W_2(A)$	
		$R_3(A)$
$W_1(B)$		
		$W_3(A)$
	$R_2(B)$	
	$W_2(B)$	



Permutando a serial

Ejemplo

Se procede a permutar:

T1	T2	T3
$R_1(B)$		
	$R_2(A)$	
	$W_2(A)$	
$W_1(B)$		
		$R_3(A)$
		$W_3(A)$
	$R_2(B)$	
	$W_2(B)$	



Permutando a serial

Ejemplo

Se procede a permutar:

T1	T2	T3
$R_1(B)$		
	$R_2(A)$	
	$W_2(A)$	
$W_1(B)$		
		$R_3(A)$
		$W_3(A)$
	$R_2(B)$	
	$W_2(B)$	

A blue double-headed arrow indicates a swap between the operation $W_2(A)$ in the second column (T2) and the operation $W_1(B)$ in the fourth column (T1). The arrow originates from $W_2(A)$ and points to $W_1(B)$.

Permutando a serial

Ejemplo

Se procede a permutar:

T1	T2	T3
$R_1(B)$		
	$R_2(A)$	
$W_1(B)$		
	$W_2(A)$	
		$R_3(A)$
		$W_3(A)$
	$R_2(B)$	
	$W_2(B)$	

A blue arrow points from the operation $W_2(A)$ in the second column (T2) to the operation $W_1(B)$ in the third column (T1), indicating a swap between these two operations in the serial schedule.

Permutando a serial

Ejemplo

Se procede a permutar:

T1	T2	T3
$R_1(B)$		
	$R_2(A)$	
$W_1(B)$		
	$W_2(A)$	
		$R_3(A)$
		$W_3(A)$
	$R_2(B)$	
	$W_2(B)$	

Permutando a serial

Ejemplo

Se procede a permutar...

T1	T2	T3
$R_1(B)$		
$W_1(B)$		
	$R_2(A)$	
	$W_2(A)$	
		$R_3(A)$
		$W_3(A)$
	$R_2(B)$	
	$W_2(B)$	

The diagram illustrates the process of permuting a schedule to achieve serializability. It shows three transactions: T1, T2, and T3. T1 has operations $R_1(B)$ and $W_1(B)$. T2 has operations $R_2(A)$, $W_2(A)$, $R_2(B)$, and $W_2(B)$. T3 has operations $R_3(A)$ and $W_3(A)$. The operations are arranged in a grid. Arrows indicate the movement of operations between transactions: $R_3(A)$ and $W_3(A)$ are moved from T3 to T2, and $R_2(B)$ and $W_2(B)$ are moved from T2 to T3. This results in a serial schedule where the operations of each transaction are contiguous.

Permutando a serial

Ejemplo

T1	T2	T3
$R_1(B)$		
$W_1(B)$		
	$R_2(A)$	
	$W_2(A)$	
	$R_2(B)$	
	$W_2(B)$	
		$R_3(A)$
		$W_3(A)$

$T1; T2; T3 \Rightarrow$ **conflict serializable** $\Rightarrow$ **serializable**

Grafo de precedencia

Serialización



Un **grafo de precedencia** es un grafo dirigido y sin ciclos donde los nodos corresponden a instrucciones (en este caso, a transacciones).

Teorema

Este proceso es exponencial ($n!$), con n el número de operaciones en todas las transacciones.

Grafo de precedencia

¿Es *conflict serializable*?

T1	T2	T3
	$R_2(A)$	
$R_1(B)$		
	$W_2(A)$	
	$R_2(B)$	
		$R_3(A)$
$W_1(B)$		
		$W_3(A)$
	$W_2(B)$	



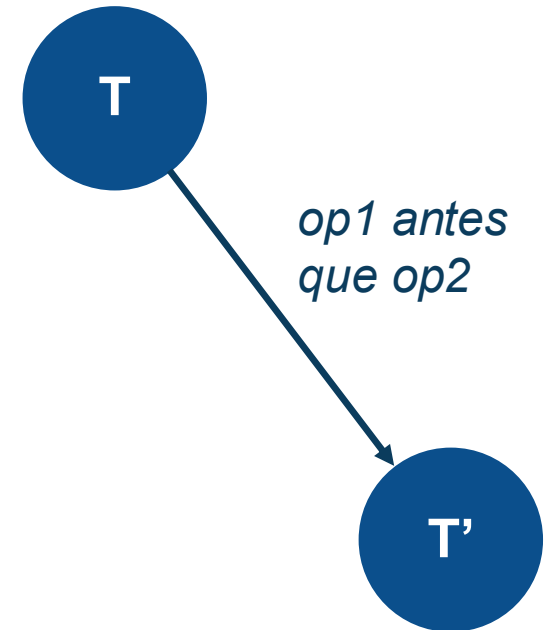
Para responder, se construye el **grafo de precedencia** del schedule.

Grafo de precedencia

Dado un **schedule**, es posible construir su grafo de precedencia:

Nodos: las transacciones del sistema.

Aristas: existe una arista de **T** a **T'** si **T** ejecuta una operación **op1** antes que una operación **op2** de **T'**, tal que **op1** y **op2** no se pueden permutar.



Grafo de precedencia

¿Es *conflict serializable*?

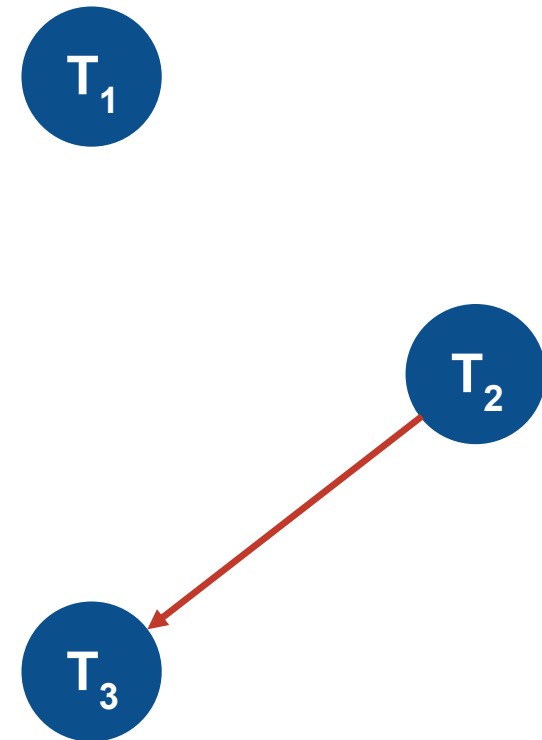
T1	T2	T3
	$R_2(A)$	
$R_1(B)$		
	$W_2(A)$	
	$R_2(B)$	
		$R_3(A)$
$W_1(B)$		
		$W_3(A)$
	$W_2(B)$	



Grafo de precedencia

¿Es *conflict serializable*?

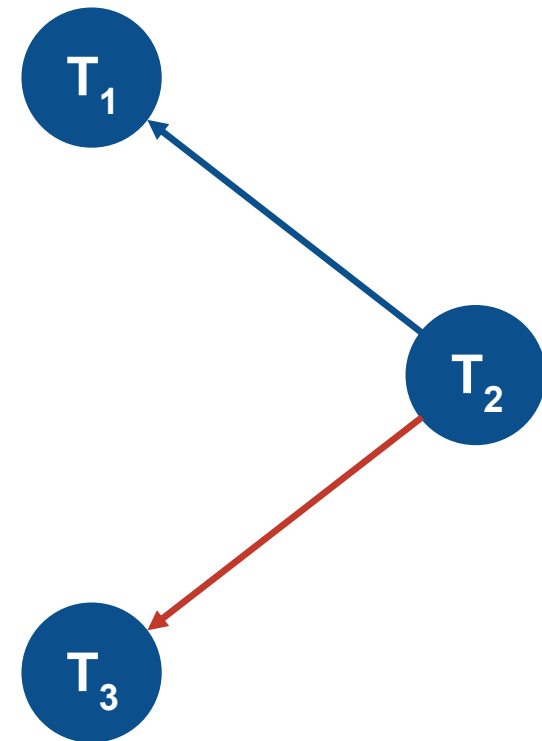
T1	T2	T3
	$R_2(A)$	
$R_1(B)$		
	$W_2(A)$	
	$R_2(B)$	
		$R_3(A)$
$W_1(B)$		
		$W_3(A)$
	$W_2(B)$	



Grafo de precedencia

¿Es *conflict serializable*?

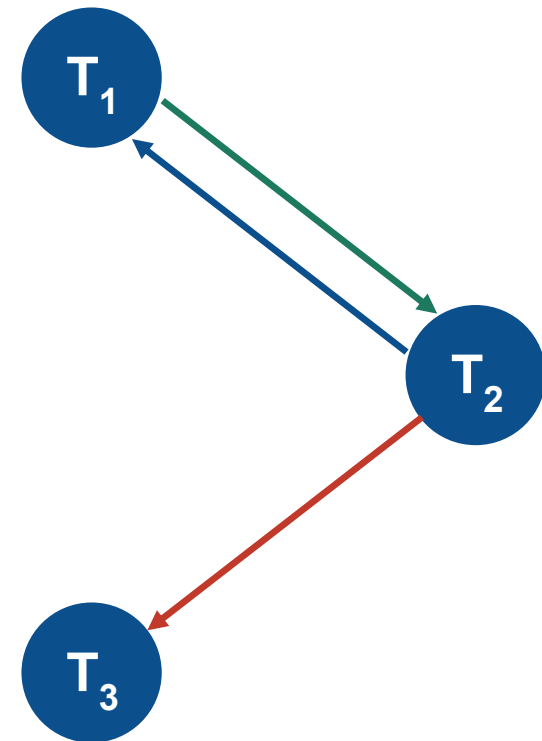
T1	T2	T3
	$R_2(A)$	
$R_1(B)$		
	$W_2(A)$	
	$R_2(B)$	
		$R_3(A)$
$W_1(B)$		
		$W_3(A)$
	$W_2(B)$	



Grafo de precedencia

¿Es *conflict serializable*?

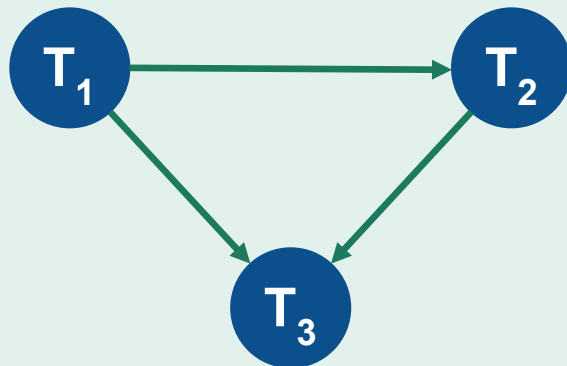
T1	T2	T3
	$R_2(A)$	
$R_1(B)$		
	$W_2(A)$	
	$R_2(B)$	
		$R_3(A)$
$W_1(B)$		
		$W_3(A)$
	$W_2(B)$	



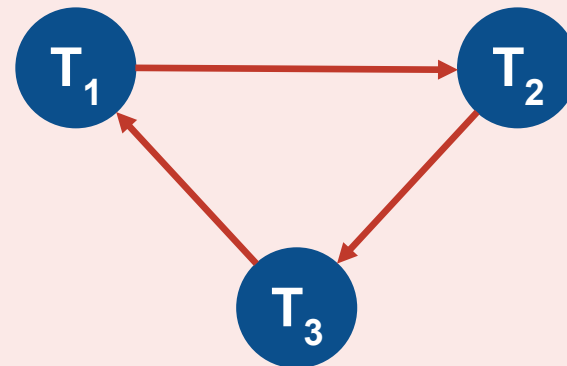
Grafo de precedencia

Teorema

Un schedule es *conflict serializable* **si y solo si** su grafo de precedencia es **acíclico**.



Acíclico $\Rightarrow$ conflict serializable

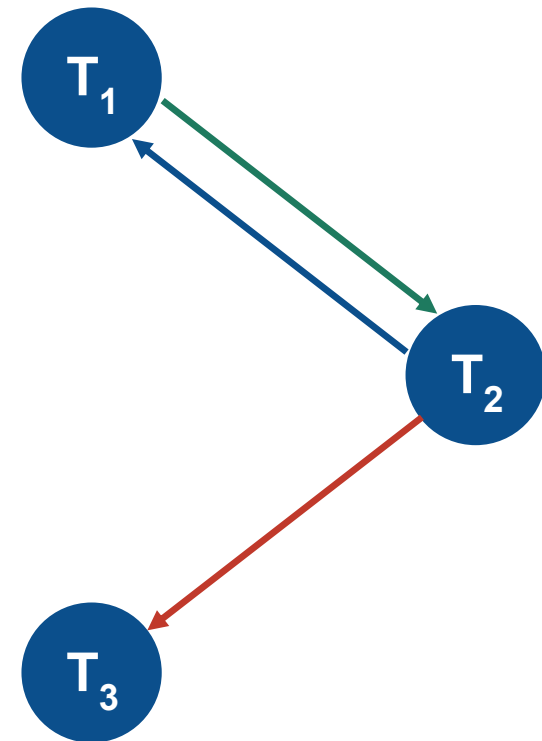


Con ciclo $\Rightarrow$ no lo es

Grafo de precedencia

¿Es *conflict serializable*?

T1	T2	T3
	$R_2(A)$	
$R_1(B)$		
	$W_2(A)$	
	$R_2(B)$	
		$R_3(A)$
$W_1(B)$		
		$W_3(A)$
	$W_2(B)$	



Ciclo $\Rightarrow$ no es conflict serializable

Locks

Strict 2PL (Two-Phase Locking)



Es el protocolo de control de concurrencia **más utilizado** en los DBMS.



Se basa en la utilización de **bloqueos (locks)**.



Se compone de **dos reglas**.

Reglas



Regla 1

Si una transacción T desea leer/modificar un objeto, primero solicita un **shared lock** / **exclusive lock** sobre el objeto.



Regla 2

Cuando la transacción se completa, libera **todos** los locks que mantenía.

Strict 2PL

Funcionamiento de los locks



Una transacción que solicita un lock **se suspende** hasta que el lock le es otorgado.



Si una transacción mantiene un **exclusive lock** sobre un objeto, ninguna otra puede mantener un shared o exclusive lock sobre él.

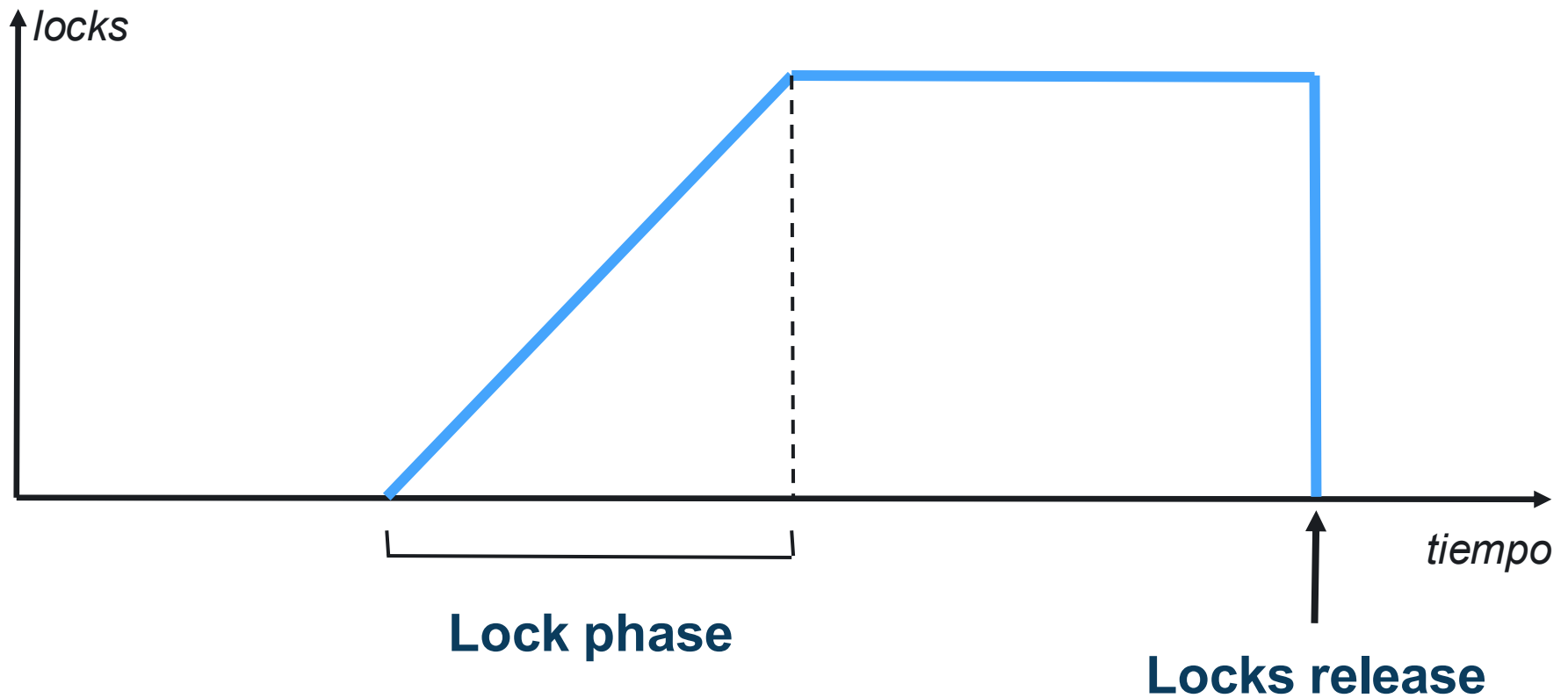


Por lo anterior, para obtener un exclusive lock, no debe existir **ningún** lock sobre el objeto.

Estas reglas aseguran solo **schedules conflict serializables**.

Strict 2PL

Locks mantenidos a lo largo del tiempo



Cómo funciona 2PL

Locks de T1:

Shared lock (A)

T1	T2
R(A)	
	W(A)
	W(B)
W(B)	

Locks de T2:

← **Espera a T1**

Cómo funciona 2PL

Locks de T1:

Shared lock (A)

Exclusive lock (B)

T1	T2
R(A)	
W(B)	
	W(A)
	W(B)

Locks de T2:

← **Espera a T1**

Cómo funciona 2PL

Locks de T1:

Listo con T1

T1	T2
R(A)	
W(B)	
	W(A)
	W(B)

Locks de T2:

← **Espera a T1**

Cómo funciona 2PL

Locks de T1:

Listo con T1

T1	T2
R(A)	
W(B)	
	W(A)
	W(B)

Locks de T2:

← **Exclusive lock (A)**

...

Problemas con locks

1. Long transactions



Son transacciones que requieren mucho tiempo para completarse debido a la complejidad de sus operaciones, al volumen de datos que manejan o a las interacciones que requieren con otros sistemas o recursos.

Este tipo de transacciones puede generar problemas significativos de **rendimiento**, **uso de recursos** y **gestión de bloqueos**, lo que afecta negativamente la concurrencia y la eficiencia del sistema.

Ejemplo: long transaction

Análisis financiero de fin de día

Un banco desea realizar un análisis financiero complejo al final del día, que involucra:



Revisar todas las transacciones del día de cada cuenta, para detectar patrones de fraude.



Calcular los intereses acumulados durante el día para todas las cuentas de ahorro.



Actualizar los saldos de cada cliente según los resultados del día.

Ejemplo: long transaction

Consecuencias



Duración

Esta operación podría tardar horas, debido a la gran cantidad de datos y a las consultas complejas necesarias.



Bloqueo

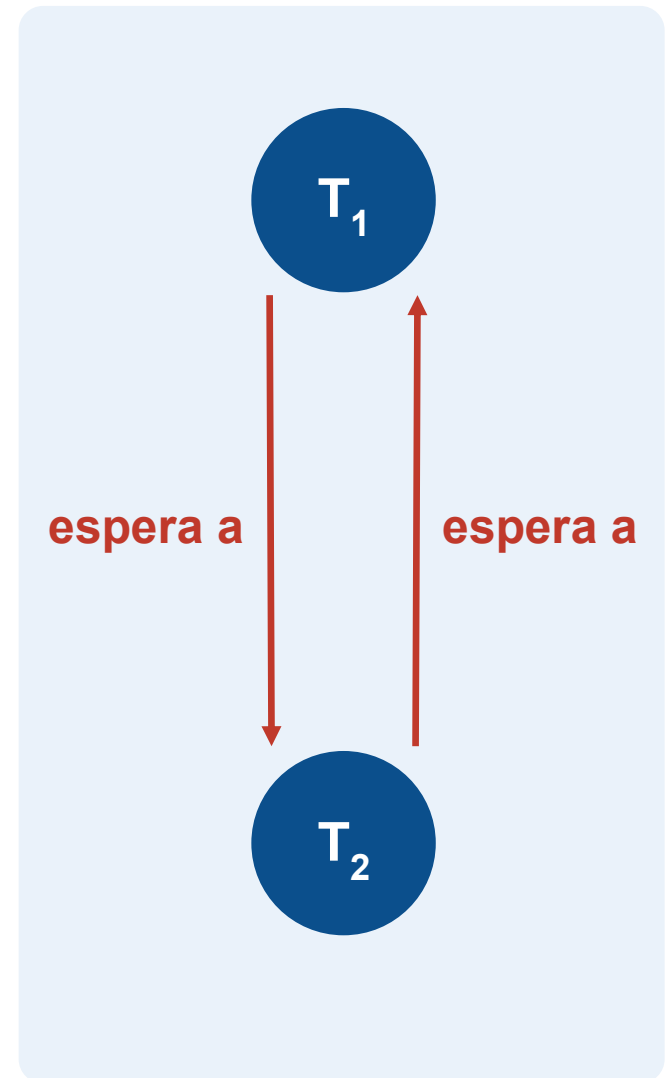
Durante ese tiempo, cualquier intento de otros usuarios o procesos de acceder o modificar los datos de las cuentas afectadas puede quedar bloqueado hasta que la transacción larga finalice.

Problemas con locks

2. Deadlocks

Ocurren cuando dos o más transacciones quedan en un **estado de espera permanente**, porque cada una posee un bloqueo sobre un recurso que la otra intenta bloquear.

Cada transacción espera que la otra libere su bloqueo, lo que crea un **ciclo de dependencias** que nunca se resuelve por sí solo.



Deadlock en 2PL

Locks de T1:

Shared lock (A)

T1	T2
R(A)	
	W(B)
	W(A)
W(B)	

Locks de T2:

Exclusive lock (B)

Espera a T2 →

← **Espera a T1**

Deadlock:

T2 espera que termine T1.

T1 espera que termine T2.



Transacciones en SQL

SQL y transacciones

```
START TRANSACTION;
UPDATE Actores
SET bio = 'El mejor actor'
WHERE nombre = 'Adrian Soto';
ROLLBACK;
```

1

2

1

Se realiza **automáticamente** al ejecutar una consulta. ¡O al conectarse a la BD desde un lenguaje de programación!

2

Para **deshacer** una transacción.

SQL y transacciones

Savepoints

```
START TRANSACTION;  
  UPDATE Actores  
  SET bio = 'El mejor actor'  
  WHERE nombre = 'Adrian Soto';  
  SAVEPOINT MejorActor;  
  UPDATE Actores  
  SET bio = 'El peor actor'  
  WHERE nombre = 'Juan Reutter';  
ROLLBACK TO SAVEPOINT MejorActor;
```

1

1

Al ejecutarse, se deshace todo lo realizado después del **SAVEPOINT**.

2

Útil en un programa que realiza varias transacciones y verifica condiciones.

Granularidad de locks

Lock a nivel de tabla

```
SELECT S.rating, MIN(S.age)
FROM Sailors AS S
WHERE S.rating = 8;
```

sid	age	rating
1	34	9
2	21	8
3	16	8
4	26	10



Lock seguro: se bloquea toda la tabla.



Lock razonable: se bloquean las tuplas con rating = 8.

Granularidad y “fantasmas”

T₁

```
SELECT S.rating, MIN(S.age)
FROM Sailors AS S
WHERE S.rating = 8;
```



sid	age	rating
1	34	9
2	21	8
3	16	8
4	26	10

shared lock

T₂

```
INSERT INTO Sailors AS S
VALUES (5, 22, 8);
```

sid	age	rating
1	34	9
2	21	8
3	16	8
4	26	10
5	22	8

shared lock (T₁)

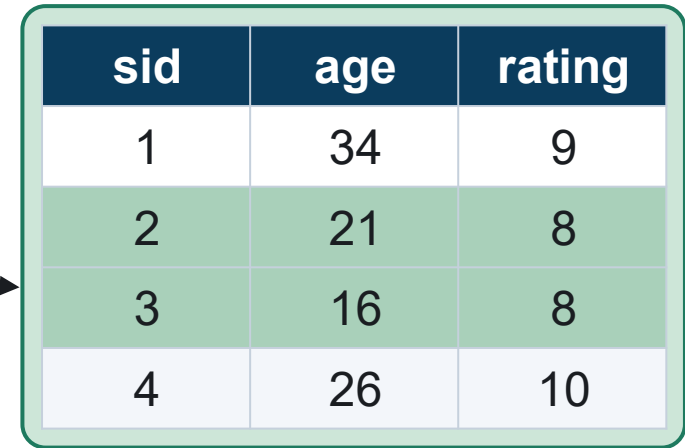
“fantasma”



Granularidad y “fantasmas”

T1

```
SELECT S.rating, MIN(S.age)
FROM Sailors AS S
WHERE S.rating = 8;
```



sid	age	rating
1	34	9
2	21	8
3	16	8
4	26	10

shared lock

T2

```
INSERT INTO Sailors AS S
VALUES (5, 22, 8);
```



espera a T1

Niveles de aislamiento

Granularidad

```
SET TRANSACTION ISOLATION LEVEL <level>  
    READ ONLY
```

```
SET TRANSACTION ISOLATION LEVEL <level>  
    READ WRITE
```

Niveles de aislamiento

Granularidad

```
SET TRANSACTION ISOLATION LEVEL <level>  
READ ONLY
```

```
SET TRANSACTION ISOLATION LEVEL <level>  
READ WRITE
```



¿Qué se puede hacer sobre las tablas en la transacción?

Niveles de aislamiento

Granularidad

```
SET TRANSACTION ISOLATION LEVEL <level>  
    READ ONLY  
SET TRANSACTION ISOLATION LEVEL <level>  
    READ WRITE
```

Level	Dirty Read	Unrepeatable Read	Phantom
SERIALIZABLE	No	No	No
REPEATABLE READ	No	No	Maybe
READ COMMITTED	No	Maybe	Maybe
READ UNCOMMITTED	Maybe	Maybe	Maybe

Lock razonable: tuplas de S con rating = 8.